

Part A – Domain Modelling and Database Design

Backend Engineer Assignment

Abstract

This document describes the relational data model for the coupon redemption system. The model focuses on correctness, explicit integrity constraints, and safe redemption under concurrent requests. The database is PostgreSQL and the schema is intended to be managed through SQL migrations.

1 Domain Overview

The system manages coupon redemption across four main domain concepts:

- **Campaigns:** business initiatives that group one or more coupons and define campaign-level availability and redemption limits.
- **Coupons:** redeemable codes associated with a campaign. Each coupon has its own status, optional expiration timestamp, and optional redemption limit.
- **Users:** actors that can redeem coupons. Users have a unique email and a role.
- **Redemptions:** immutable records of successful coupon redemptions by users.

The model intentionally keeps redemption events in a dedicated table instead of relying only on counters. Counters are useful for efficient limit checks, while redemption records provide traceability and allow the database to enforce that a user cannot redeem the same coupon twice.

2 Entity Relationship Schema

Figure 1 shows the relational structure of the system.

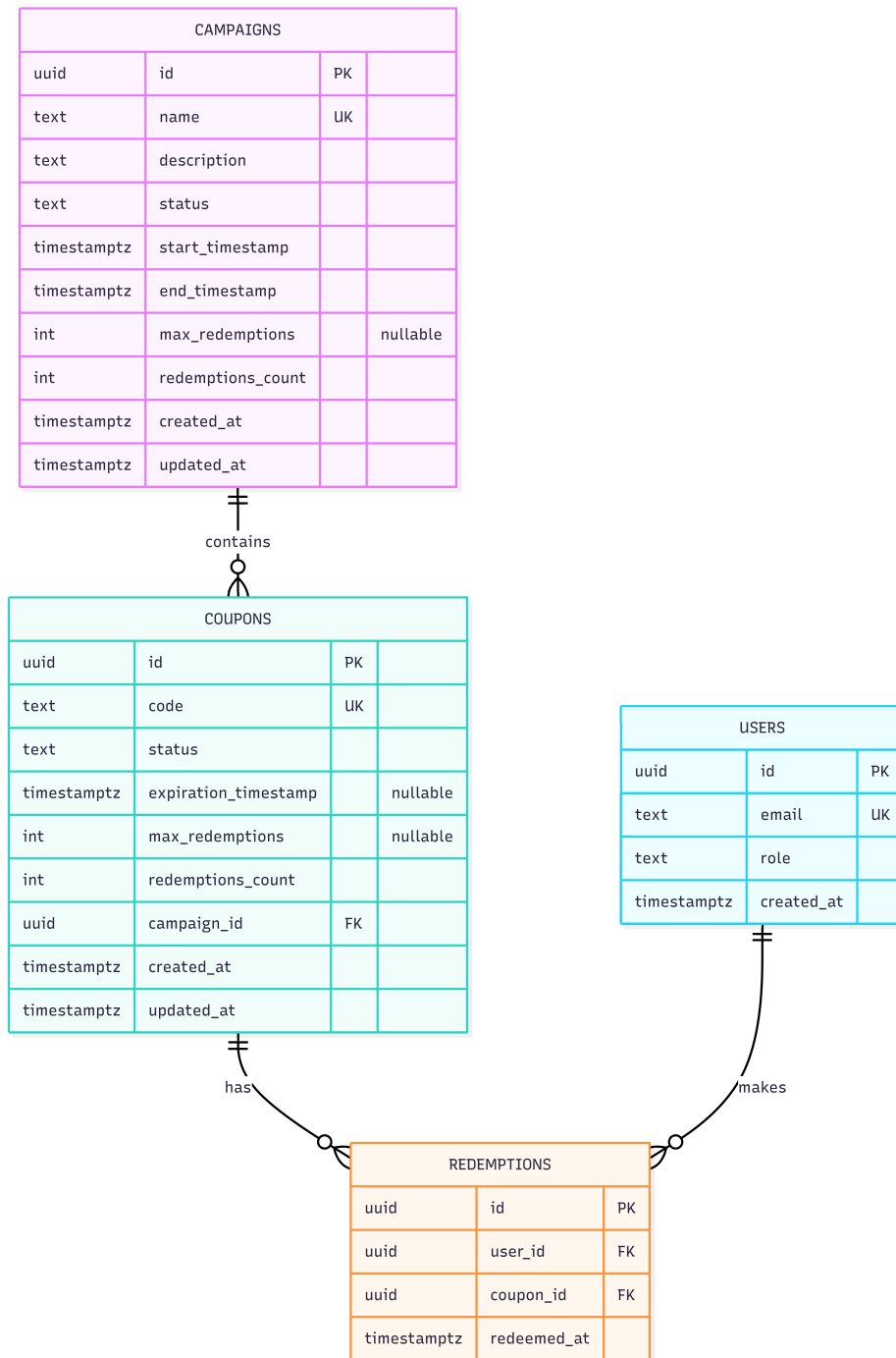


Figure 1: Entity relationship schema for campaigns, coupons, users, and redemptions.

3 Relations and Cardinalities

The schema contains the following relationships:

- **Campaigns to Coupons:** one campaign can contain zero or many coupons, while each coupon belongs to exactly one campaign. This is represented by the foreign key `coupons.campaign_id`.
- **Coupons to Redemptions:** one coupon can have zero or many redemption records, while each redemption refers to exactly one coupon. This is represented by the foreign key `redemptions.coupon_id`.
- **Users to Redemptions:** one user can make zero or many redemptions, while each redemption belongs to exactly one user. This is represented by the foreign key `redemptions.user_id`.

The `redemptions` table also models the many-to-many relationship between users and coupons: a user can redeem many different coupons, and a coupon can be redeemed by many different users. However,

the pair (user_id, coupon_id) must be unique, preventing double redemption of the same coupon by the same user.

4 Relational Schema

4.1 Campaigns

Column	Type	Constraint / Meaning
id	uuid	Primary key
name	text	Not null, unique
description	text	Nullable
status	text	available or not-available
start_timestamp	timestampz	Not null
end_timestamp	timestampz	Not null
max_redemptions	integer	Nullable; null means unlimited
redemptions_count	integer	Not null, default 0
created_at	timestampz	Not null, default current timestamp
updated_at	timestampz	Not null, default current timestamp

4.2 Coupons

Column	Type	Constraint / Meaning
id	uuid	Primary key
code	text	Not null, unique
status	text	available or not-available
expiration_timestamp	timestampz	Nullable; null means no coupon-specific expiration
max_redemptions	integer	Nullable; null means unlimited
redemptions_count	integer	Not null, default 0
campaign_id	uuid	Foreign key to campaigns(id)
created_at	timestampz	Not null, default current timestamp
updated_at	timestampz	Not null, default current timestamp

4.3 Users

Column	Type	Constraint / Meaning
id	uuid	Primary key
email	text	Not null, unique
role	text	user or admin
created_at	timestampz	Not null, default current timestamp

4.4 Redemptions

Column	Type	Constraint / Meaning
id	uuid	Primary key
user_id	uuid	Foreign key to users(id)
coupon_id	uuid	Foreign key to coupons(id)
redeemed_at	timestampz	Not null, default current timestamp

5 Integrity Constraints

The model relies on database constraints rather than application checks alone. The most relevant constraints are:

- `coupons.code` is unique, because coupon redemption is addressed by coupon code.
- `users.email` is unique, as required by the domain model.
- `campaigns.name` is unique and is used to detect an already existing campaign in `POST /coupons`.

- `redemptions(user_id, coupon_id)` is unique to prevent the same user from redeeming the same coupon more than once.
- Foreign keys preserve referential integrity between campaigns, coupons, users, and redemptions.
- Status columns are constrained to the accepted values: `available` and `not-available`.
- User role is constrained to `user` or `admin`.
- Text identifiers such as campaign name, coupon code, and user email cannot be blank.
- Campaign time windows must be valid: `end_timestamp >= start_timestamp`.
- Redemption counters cannot be negative.
- If `max_redemptions` is not null, `redemptions_count` cannot exceed it.

5.1 Foreign-Key Referential Actions

The schema makes the deletion and update semantics explicit for every foreign key:

Source column	Target relation	ON DELETE	ON UPDATE
<code>coupons.campaign_id</code>	<code>campaigns(id)</code>	RESTRICT	CASCADE
<code>redemptions.user_id</code>	<code>users(id)</code>	RESTRICT	CASCADE
<code>redemptions.coupon_id</code>	<code>coupons(id)</code>	RESTRICT	CASCADE

ON DELETE RESTRICT is intentional. A campaign cannot be deleted while coupons still belong to it, and users or coupons cannot be deleted while redemption history depends on them. This avoids accidental loss of business history through implicit cascading deletes. If the product later requires removal, it should be handled through an explicit archival or domain-specific cleanup policy.

ON UPDATE CASCADE is also explicit. Primary UUID identifiers are not expected to change during normal operation, but if an identifier is corrected during controlled maintenance or data migration, dependent rows remain consistent automatically instead of becoming orphaned.

The migration expresses those referential actions explicitly:

```

CONSTRAINT coupons_campaign_id_fk FOREIGN KEY (campaign_id)
REFERENCES campaigns(id)
ON UPDATE CASCADE
ON DELETE RESTRICT,

CONSTRAINT redemptions_user_id_fk FOREIGN KEY (user_id)
REFERENCES users(id)
ON UPDATE CASCADE
ON DELETE RESTRICT,

CONSTRAINT redemptions_coupon_id_fk FOREIGN KEY (coupon_id)
REFERENCES coupons(id)
ON UPDATE CASCADE
ON DELETE RESTRICT

```

An example SQL representation of the most important constraints is:

```

CONSTRAINT campaigns_name_unique UNIQUE (name),
CONSTRAINT coupons_code_unique UNIQUE (code),
CONSTRAINT users_email_unique UNIQUE (email),
CONSTRAINT redemptions_user_coupon_unique UNIQUE (user_id, coupon_id),

CONSTRAINT campaigns_status_check
CHECK (status IN ('available', 'not-available')),
CONSTRAINT coupons_status_check
CHECK (status IN ('available', 'not-available')),
CONSTRAINT users_role_check
CHECK (role IN ('user', 'admin')),

CONSTRAINT campaigns_time_window_check
CHECK (end_timestamp >= start_timestamp),

```

```

CONSTRAINT campaigns_name_not_blank
    CHECK (btrim(name) <> ''),
CONSTRAINT coupons_code_not_blank
    CHECK (btrim(code) <> ''),
CONSTRAINT users_email_not_blank
    CHECK (btrim(email) <> ''),

CONSTRAINT campaigns_redemptions_count_check
    CHECK (redemptions_count >= 0),
CONSTRAINT coupons_redemptions_count_check
    CHECK (redemptions_count >= 0),
CONSTRAINT campaigns_redemptions_limit_check
    CHECK (max_redemptions IS NULL OR redemptions_count <= max_redemptions),
CONSTRAINT coupons_redemptions_limit_check
    CHECK (max_redemptions IS NULL OR redemptions_count <= max_redemptions)

```

6 SQL Migration Details

The SQL migration also includes a few implementation-level details that support the relational model:

- `pgcrypto` is enabled so primary keys can default to `gen_random_uuid()`.
- Listing-oriented indexes are created for campaign and coupon filtering.
- Foreign-key indexes are created for joins and redemption lookups.
- `updated_at` is maintained through PostgreSQL triggers on `campaigns` and `coupons`.

7 Design Decisions

7.1 Campaign Identity

The assignment requires defining how an already existing campaign is detected when creating a coupon. This design uses `campaigns.name` as a unique natural key. If a campaign with the same name already exists, the API reuses it and creates only the new coupon.

This is a pragmatic choice for the assignment: it is easy to understand, enforceable at database level, and sufficient for the required endpoint. In a larger production system, a separate external campaign identifier could be preferable.

7.2 Expiration Semantics

The model uses explicit timestamp rules:

- A campaign is expired when `end_timestamp < now()`.
- A coupon is expired when `expiration_timestamp IS NOT NULL AND expiration_timestamp < now()`.
- A null coupon expiration means the coupon has no coupon-specific expiration, but it is still bounded by its campaign validity.
- Future campaigns are not expired. They are included in the admin listing if status and expiration rules allow it, but they are not redeemable before `start_timestamp <= now()`.

7.3 Redemption Counters

Both campaigns and coupons store `redemptions_count`. These counters allow efficient checks against `max_redemptions`. They are not updated independently from redemption records: the redemption insert and both counter increments happen in the same database transaction.

7.4 Concurrency Safety

Redemption is the critical operation. The implementation uses a short PostgreSQL transaction:

1. Select the target coupon and its campaign using row-level locks, for example `SELECT ... FOR UPDATE`.
2. Validate status, expiration timestamps, start timestamp, and redemption limits while the rows are locked.
3. Insert the redemption record.
4. Increment both `coupons.redemptions_count` and `campaigns.redemptions_count`.
5. Commit the transaction.

The row locks serialize concurrent redemptions for the same coupon and campaign. The unique constraint on `redemptions(user_id, coupon_id)` remains the final database-level protection against duplicate redemption by the same user. Any validation failure, insert conflict, or unexpected database error rolls back the full unit of work, so the redemption row and both counters never diverge.

At a high level, the pattern is:

```
BEGIN;

SELECT ...
FROM coupons
INNER JOIN campaigns ON campaigns.id = coupons.campaign_id
WHERE coupons.code = $1
FOR UPDATE OF coupons, campaigns;

-- Validate state and limits while the relevant rows are locked.
INSERT INTO redemptions (user_id, coupon_id)
VALUES ($2, $3);

UPDATE coupons
SET redemptions_count = redemptions_count + 1
WHERE id = $3;

UPDATE campaigns
SET redemptions_count = redemptions_count + 1
WHERE id = $4;

COMMIT;
```

The API uses parameterized SQL statements for these operations. Request values are passed as bound parameters such as `$1`, `$2`, and `$3`, rather than being concatenated into SQL text.